CHAPTER 1

DESIGNING LINKEDIN

1 Getting Ready: The LinkedIn System 2
2 Requirements for LinkedIn 3
3 Use Case Diagram for LinkedIn . . . 4
4 Class Diagram for LinkedIn 7
5 Sequence Diagram for LinkedIn . . . 19
6 Activity Diagram for LinkedIn 19
7 Code for LinkedIn 20
8 Summary: Designing LinkedIn 29
9 Key Insights and Tips: LinkedIn System 32

This chapter explores how to design the LinkedIn system, covering requirements, UML diagrams, and implementation code for a complete design.

1 Getting Ready: The LinkedIn System

1.1 Problem definition

LinkedIn is a professional networking platform that helps users manage their professional identity. It enables users to connect with trusted professionals, grow their careers, and explore new opportunities.
The platform enables individuals to create comprehensive professional profiles, featuring their education, experience, skills, achievements, and endorsements. Users can also create posts, interact with others' content, join or create groups, and manage connections.
In addition to personal profiles, LinkedIn supports company pages, where organizations can list job openings and recruit candidates. Users can search for other professionals, send connection requests, exchange messages, and receive notifications about relevant activity.
LinkedIn is very similar to Facebook in terms of its layout and design. However, its features are more specialized because it caters to professionals. If we are familiar with using Facebook or any similar social network, we may find LinkedIn familiar.

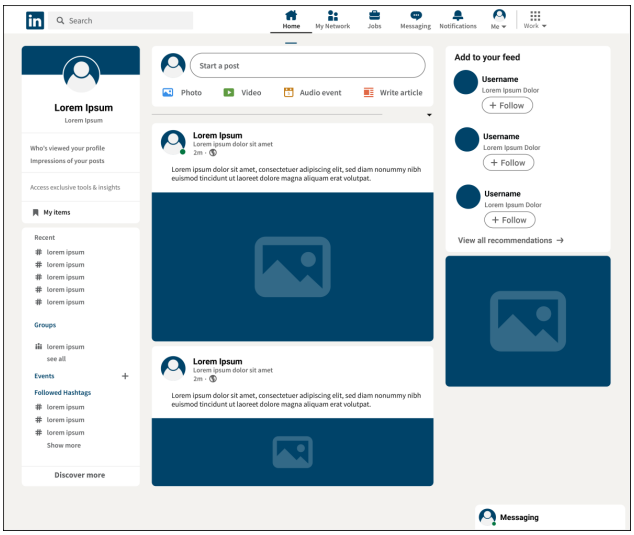


Figure 1.1: The home page of a LinkedIn user

1.2 Expectations from the interviewee

LinkedIn offers a range of functionalities to its users. As our focus is limited to a specific set of features, it is essential to narrow down the components included in the LinkedIn design. The following sections highlight some key areas that the interviewer may expect you to explore during the system design discussion.

Discoverability

Understanding how users search for content and other users is important. The relevant questions include:

- How are users able to search other users' profiles?
- Can users search for groups and pages?

Connections and following

Both connections and following are the primary features of LinkedIn. Make sure to ask the following questions of the interviewer:

- How are users able to connect with other users?

- How can users follow or unfollow other users or company pages without needing to connect?

Groups, pages, and jobs

Groups and pages on LinkedIn create a space for people looking for similar job opportunities. Make sure to define your requirements. You may ask the following questions of the interviewer:

- Can users create both groups and company pages?
- Can users join a group?
- Can users follow company pages?
- Can company pages post jobs that users can apply for?

Alerts

Notifications and alerts enable users to stay informed about activity within their circle. Therefore, you may want to understand how alerts work in your system. You may ask the following questions:

- How will users be notified about messages, connection requests, or comments on their posts?

1.3 Design approach

We'll follow a bottom-up design approach to build the LinkedIn system. The steps include:

- Design the smallest units first (e.g., posts, comments, users, connections).
- Combine these to build larger components (e.g., groups, company pages, profiles).
- Continue layering until the complete LinkedIn platform is modeled.

1.4 Design pattern

During an interview, it is always a good practice to discuss the design patterns that LinkedIn falls under. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design the LinkedIn system? Please elaborate on your choice(s).

.

Let's explore the requirements of the LinkedIn design in the next lesson.

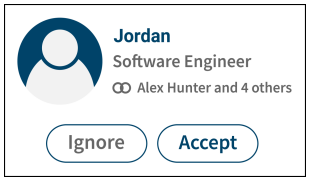
2 Requirements for LinkedIn

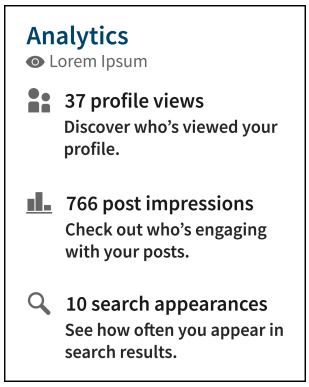
In this lesson, we'll list the requirements of the LinkedIn design problem. This is a crucial step because requirements define the scope of a problem. Therefore, accurately obtaining and thoroughly understanding them from the interviewer will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label, "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirements collection

The requirements for LinkedIn are defined below:

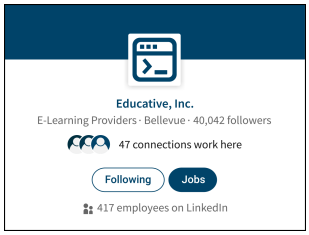
R1: Users should be able to add information to their profiles, including education, experiences, achievements, and skills.
R2: Users should be able to search for and view pages, groups, and other users.
R3: Users should be able to send and cancel connection requests and respond to other users' connection requests by either accepting or ignoring them.
R4: Users should be able to follow other users without needing to add them as a connection.
R5: Users should be able to view their number of connections, profile views, post impressions, and search appearances.
R6: Users should be able to request and give recommendations to other users.





R7: Users should be able to write a new post.
R8: Users should be able to react, share, and comment on a post and on an existing comment.
R9: A user should be able to send and receive messages from other users.

R10: The system should notify the user about messages, connection requests, search appearances, or comments on their posts.
R11: Users should be able to create company pages and follow other company pages.
R12: Company pages can list job openings that users can apply for.
R13: A user should be able to create and join groups.



We've identified our requirements for the problem. In the next lesson, we will define different use cases of LinkedIn.

3 Use Case Diagram for LinkedIn

Let's build LinkedIn's use case diagram and understand the relationship between its different components. First , we'll define the different elements of our LinkedIn system, followed by the complete use case diagram.

3.1 System

Our system is LinkedIn.

3.2 Actors

Now, we'll define the main actors of LinkedIn.

Primary actors

- **User:** This actor can create a profile including their professional information. They can create posts, apply for jobs, follow pages, and join groups. They can also interact with other users by sending them connection invitations and messages, commenting on their posts, and so on.

Secondary actors

- **System:** This sends notifications for new connection requests, messages, comments, posts, recommendations, and other relevant events.

3.3 Use cases

In this section, we'll define LinkedIn's use cases. We have listed them in order of their respective interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

User

- **Add/update profile:** To add information like education, experience, skills, and achievements to update an existing profile.
- **Search:** To find and view other users, company pages, or groups.
- **Send connection request:** To send a connection request to another user.
- **Cancel connection request:** To withdraw a previously sent connection request.
- **Accept/ignore connection request:** To accept or ignore a received connection request.
- **Follow/unfollow users:** To follow or unfollow other users without connecting.
- **View analytics:** To see the number of connections, profile views, post impressions, and search appearances.
- **Request recommendation:** To send a recommendation request to another user.
- **Give recommendation:** To write and give a recommendation to another user.
- **Create post:** To publish a new post on their profile or feed.
- **React/share/comment on the post:** To engage with a post by reacting, sharing, or commenting.
- **React/comment on comment:** To react to or comment on an existing comment.
- **Send/receive messages:** To exchange private messages with other users.
- **Receive notifications:** To be notified about messages, connection requests, or comments on posts.
- **Create company page:** To create a company page on behalf of an organization.

- **Follow the company page:** To follow a company page to receive updates.
- **Create group:** To start a new group for a community or professional interest.
- **Join a group:** To become a member of an existing group.
- **Apply to the job:** To apply for a job listed on a company page.
- **List job openings in the company page:** To list the new jobs on a company page.

System

- **Send notifications:** To inform users about new messages, connection requests, or comments on their posts.
- **Add recommendation:** To add a recommendation to the user's profile.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Generalization

- The user can add or update their profile by updating their education, experience, skills, or achievements. This indicates that the "Add/update profile" use case has a generalization relationship with the "Add/update education," "Add/update experience," "Add/update skill," and "Add/update achievement" use cases.
- The user can view their analytics by accessing their profile, where they can view their profile views, post impressions, search appearances, and connection count. This shows that the "View analytics" use case has a generalization relationship with "View profile views," "View post impressions," "View search appearances," and "View connections count" use cases.

Associations

The table below illustrates the association between actors and their corresponding use cases.

User	System
Add/update profile	Send notifications
Search	Add recommendation
Send connection request	
Cancel connection request	
Accept/ignore connection request	
Follow/unfollow users	
View analytics	
Request recommendation	
Give recommendation	
Create post	
React/share/comment on post	
React/comment on comment	
Send/receive messages	
Receive notifications	
Create company page	
Follow company page	
Create group	
Apply to job	
List job openings in company page	

Include

When a user sends a message to another user, the system notifies the recipient about the new message.

- The "Send message" use case includes "Send notification."

When a user sends a connection request, the system notifies the recipient that someone wants to connect.

- The "Send connection request" use case includes "Send notification."

When a user comments on a post, the system notifies the post owner about the new comment.

- The "Comment on post" use case includes "Send notification."

When a user reacts to a post, the system notifies the post owner about the new reaction.

- The "React to post" use case includes "Send notification."

When a user performs a search, they may want to look for job listings, company pages, or groups. Additionally , if a user searches for another user, the user who appears in the search results gets notified.

- The "Search" use case has an *include* relationship with the "Search job," " Search company," and "Search group."
- It also has an *include* relationship with "Send notification."

When a user requests recommendations, the system adds the recommendations for the user.

- The "Request recommendation" use case includes "Add recommendation."

Extend

- When users update their profiles, they may choose to add or update specific sections such as their education, experience, skills, or achievements. Therefore, the "Add/update profile" use case has an *extend* relationship with the "Add/update education," "Add/update experience," " Add/update skill," and "Add/update achievement"use cases.
- When users view their profile analytics, they can optionally view specific metrics such as profile views, post impressions, search appearances, or the number of their connections. Therefore, the "View analytics" use case has an *extend* relationship with the "View profile views," "View post impressions," " View search appearances," and "View connections count" use cases.

3.5 Use case diagram

Here's the use case diagram of LinkedIn:

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for LinkedIn

In this lesson, we'll identify and design classes, abstract classes, and interfaces based on the requirements gathered from the interviewer through our LinkedIn system.

4.1 Components of LinkedIn

As mentioned, we will design the LinkedIn system using a bottom-up approach.

Figure 1.2: The use case diagram of LinkedIn

Education, experience, and skill

The `Education`, `Experience`, `Achievement`, and `Skill` classes provide relevant information and comprise the `Profile` class.
These classes are represented below:

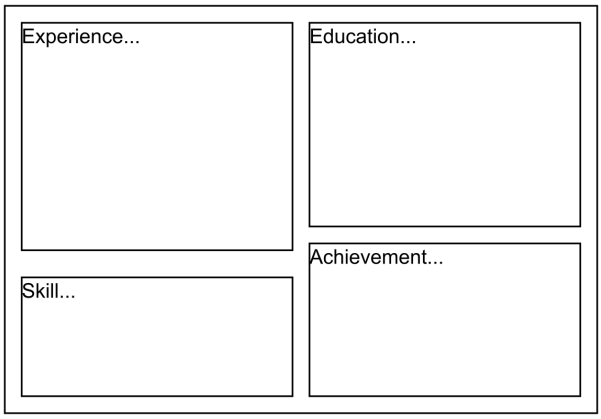


Figure 1.3: The Education, Achievement, Experience, and Skill classes

R1: Users should be able to add information to their profiles, including education, experiences, achievements, and skills.

Recommendation, achievement, and analytics

The `Recommendation`, and `Analytics` classes are used to provide relevant information and make up the `Profile` class.
These classes are represented below:

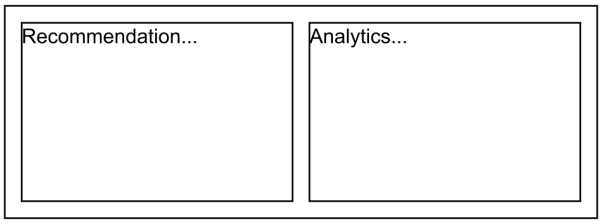


Figure 1.4: The Recommendation and Analytics classes

R5: Users should be able to view their number of connections, profile views, post impressions, and search appearances.
R6: Users should be able to request and give recommendations to other users.

Profile

The `Profile` class contains the personal information of a LinkedIn user, including their profile, cover photo, work experiences, educational details, skills, and achievements.
The visual representation of the `Profile` class is given below:

R1: Users should be able to add information to their profiles, including education, experiences, achievements, and skills.

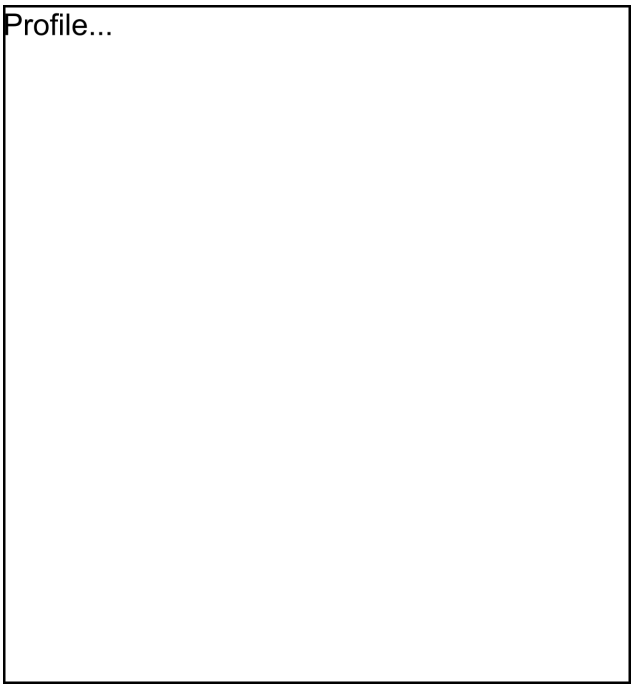


Figure 1.5: The Profile class

Company page, job, and group

The `CompanyPage` class represents a particular page for a company that exists on LinkedIn. It contains the name, description, company size, and other relevant details.
The `Job` class contains information about a job posting at a specific company, as well as current job openings.
The `Group` class represents a particular group that exists on the LinkedIn platform. This will contain the name, description, total number of users, and a unique ID to identify the group.
The representation of these classes is given below:

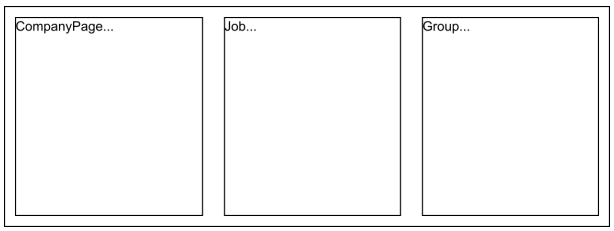


Figure 1.6: The CompanyPage, Job, and Group classes

R11: Users should be able to create company pages and follow other company pages.
R12: Company pages should list job openings that users can apply for.
R13: A user should be able to create and join groups.

Post, comment, and message

The `Post` class indicates a post made by a user. It contains information regarding its content, the number of reacts and shares of the post, and the owner of the post.
The `Comment` class indicates a comment made by a user on a post. It includes information regarding its content, the number of reacts, and the owner of the comment.
The `Message` class represents the message sent by a user to other users. Therefore, it contains content and

multimedia elements, such as images or videos.
The representation of the classes is shown below:

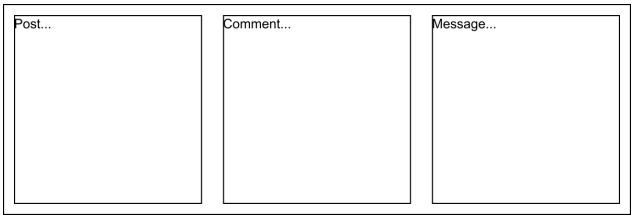


Figure 1.7: The Post, Comment, and Message classes

R7: Users should be able to write a new post.
R8: Users should be able to react, share, and comment on a post and an existing comment.
R9: A user should be able to send and receive messages from other users.

Connection invitation

The `ConnectionInvitation` class describes the details of a connection invitation. It will contain the status of the connection invite and the methods that indicate whether the request was accepted or ignored.
The class diagram of the `ConnectionInvitation` class is shown below:

Figure 1.8: The ConnectionInvitation class

R3: Users should be able to send and cancel connection requests and respond to other users' connection requests by either accepting or ignoring them.

Person

The `Person` is an abstract class that contains details like the name, address, and phone number. It is derived from the `User` class.
User
The `User` class is the main class of LinkedIn. It is responsible for various operations, such as applying for jobs, sending messages or friend requests to other users, creating posts and pages, joining groups, and numerous other tasks.
These two classes are represented below:

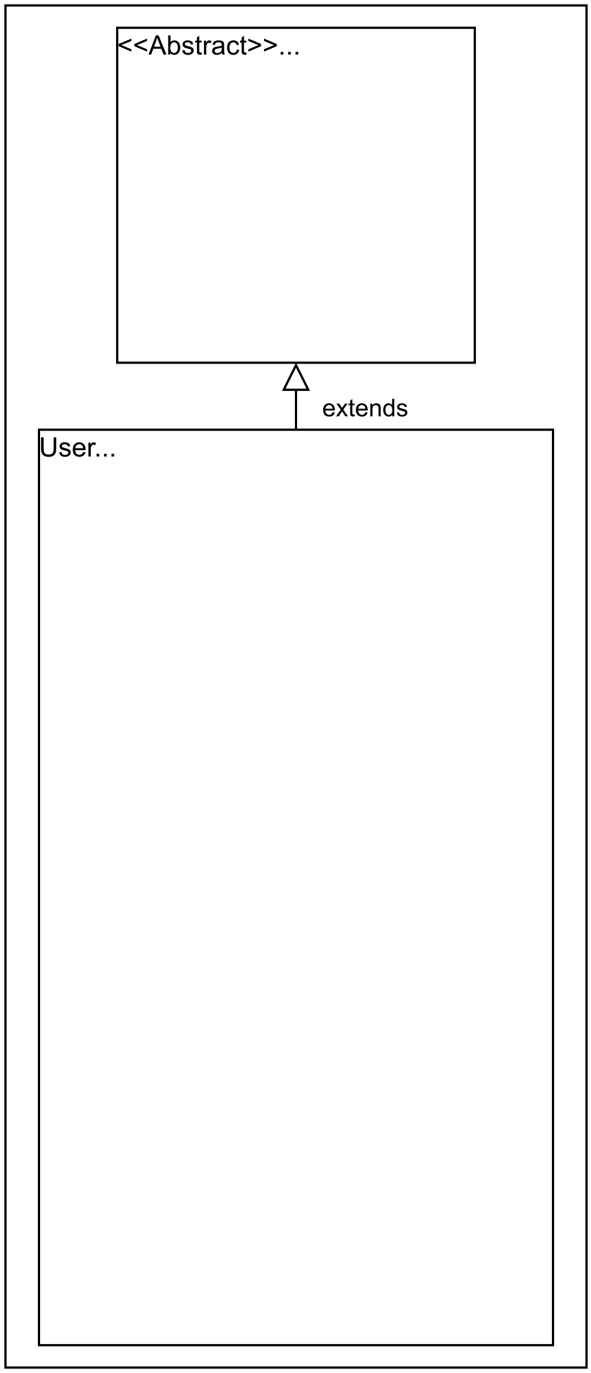


Figure 1.9: The Person and its derived classes

Notification

The `Notification` class sends a notification using the built-in notification option. It is mainly responsible for sending notifications whenever any of the following conditions are met:

- A connection invitation is received.
- A new message is received.
- A new comment is added to a post that a user has created or is following.
- A new post has been created by a user or a company page that another user is following.
- A user appeared in a search result.

The UML representation of the class is shown below:

Figure 1.10: The Notification class

R10: The system should notify the user to inform them about messages, connection requests, search appearances, or comments on their post. or comments on their post.

Search interface and search catalog

The `Search` class enables users to search for specific users, companies, groups, or job openings. It also returns the list of respective components.
The representation of the `Search` class is given below:

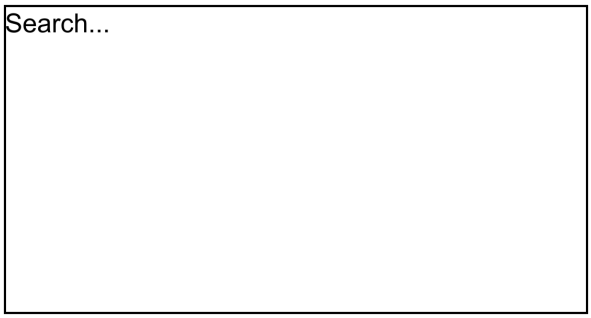


Figure 1.11: The SearchCatalog class and the Search interface

R2: Users should be able to search for and view pages, groups, and other users.

Enumerations

The enumerations required in the LinkedIn design are provided below:

- `ConnectionInviteStatus`: The connection invite status describes the status of a connection invitation—whether it is pending, accepted, or ignored.
- `JobStatus`: The job status describes the current status of a job posting—whether it is open, on hold, or closed.

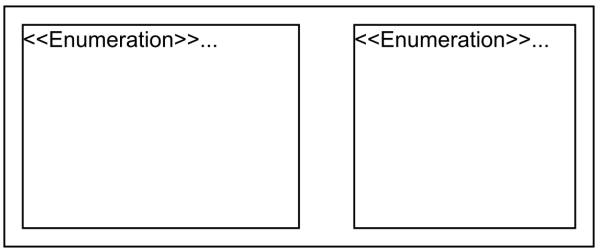


Figure 1.12: Enums in LinkedIn

Custom data type

We need to create a custom data type, `Address`, which stores the location of a LinkedIn user.

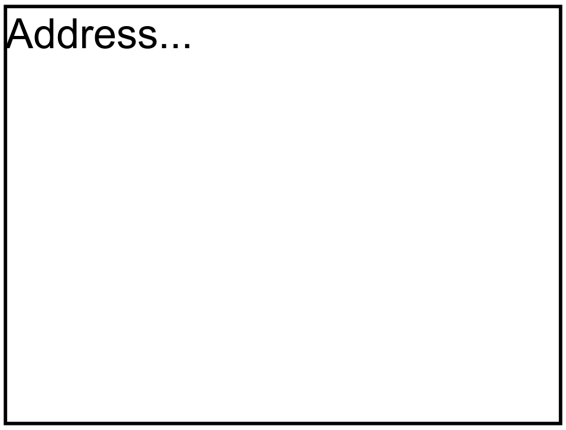


Figure 1.13: The Address custom data type

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our LinkedIn system.

Association

The class diagram has the following association relationships:

- The `User` class has a one-way association with the following:

  - Itself
  - The `CompanyPage`, `Group`, `Job`, `Post`, `Comment`, `Message`, and `ConnectionInvitation` classes
  - The `Search` class

- The `Notification` class has a one-way association with the `Message`, `Post`, `Comment`, `ConnectionInvitation`, `search`, and `Recommendation` classes.

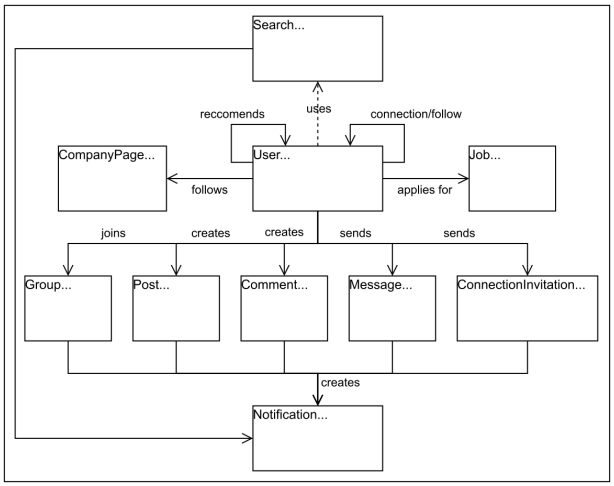


Figure 1.14: The association relationship between classes

Composition

The class diagram has the following composition relationships:

- The `Person` class is composed of the `Account` class.
- The `User` class is composed of the `Profile` class.
- The `Profile` class comprises the `Experience`, `Education`, `Skill`, `Achievement`, `Recommendation`, and `Analytics` classes.
- The `Post` class is composed of the `Comment` class.
- The `CompanyPage` class is composed of the `Job` class.

Inheritance

The following classes demonstrate an inheritance relationship:

- The `User` classes extend the `Person` class.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

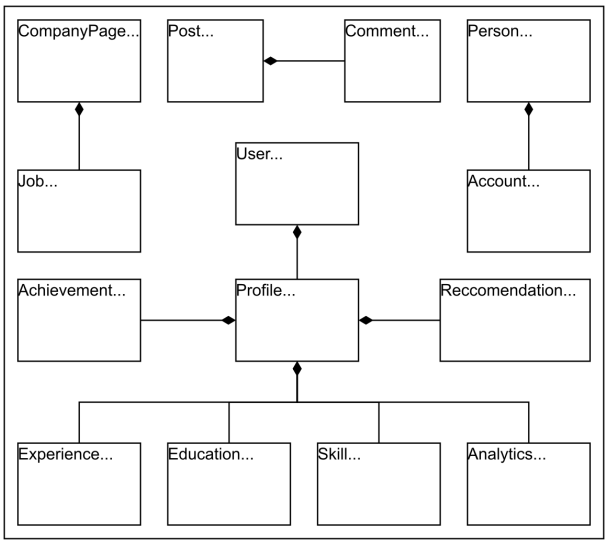


Figure 1.15: The composition relationship between classes

4.3 Class diagram of LinkedIn

In this section, we outline the multiplicity (cardinality) relationships between the main classes in our LinkedIn design system. For each relationship, we explain the allowed number of instances on each side and the real-world or design rationale behind the connection. Understanding these relationships is crucial for modeling how different entities interact and collaborate to support key workflows within the system.

Source	**Target**	**Multiplicity**	**Explanation**
Person	User	1 – 1	User extends Person
User	Profile	1 – 1	Each user has exactly one profile
User	CompanyPage	1 – 0..*	A user can create multiple company pages
User	User	0..* – 0..*	Users can follow or connect with many other users
User	Group	1 – 0..*	A user can join or create multiple groups
User	Comment	1 – 0..*	A user can make multiple comments
User	Message	1 – 0..*	A user can send or receive multiple messages
User	Post	1 – 0..*	A user can create multiple posts
User	Job	1 – 0..*	A user can apply to multiple jobs
User	ConnectionInvitation	1 – 0..*	A user can send or receive multiple connection invitations
Profile	Analytics	1 – 1	A profile has one set of analytics data
Profile	Experience	1 – 0..*	A profile can include multiple experiences

Profile	Education	1 – 0..*	A profile can include multiple education entries
Profile	Skill	0..* – 0..*	A profile can list multiple skills
Profile	Achievement	1 – 0..*	A profile can include multiple achievements
Profile	Recommendation	1 – 0..*	A profile can contain multiple recommendations
CompanyPage	Job	1 – 0..*	A company page can list multiple jobs
Job	User	1 – 1	One user posts a job
Job	Address	1 – 1	A job has one location
Job	CompanyPage	1 – 1	A job belongs to one company page
Comment	Post	0..* – 1	A post can have multiple comments
Comment	User	1 – 1	One user writes each comment
Post	Comment	1 – 0..*	A post can have multiple comments
Message	User	1 – 0..*	A message can have one sender and multiple recipients
Group	User	1 – 0..*	A group can have many members (users)
ConnectionInvitation	User	1 – 1	An invitation is between exactly two users
Notification	User	1 – 1	Each notification is sent to one user
Search	User / CompanyPage / Job / Group	1 – 0..*	A search can include users, companies, jobs, or groups

Here's the complete class diagram for LinkedIn:

4.4 Design pattern

The following design patterns have been used in the class diagram:

- We know that users can create various types of objects, such as connection invitations, posts, comments, and job listings. To centralize and manage the creation of these objects, we can use theFactory design pattern.
- We understand that users need to be notified when specific actions occur, such as receiving a connection request, appearing in a search result, or receiving a comment on a post. To handle this automatic notification mechanism, we can use the Observer design pattern.
- We recognize that certain services, such as search and notification, should utilize a single, shared instance across the entire system. To enforce that only one instance of such a service exists, we can use the Singleton design pattern.
- We know that users can search for various entities, such as users, jobs, companies, or groups, and the search logic may differ for each. To encapsulate these different search behaviors and allow them to be swapped easily, we can use the Strategy design pattern.

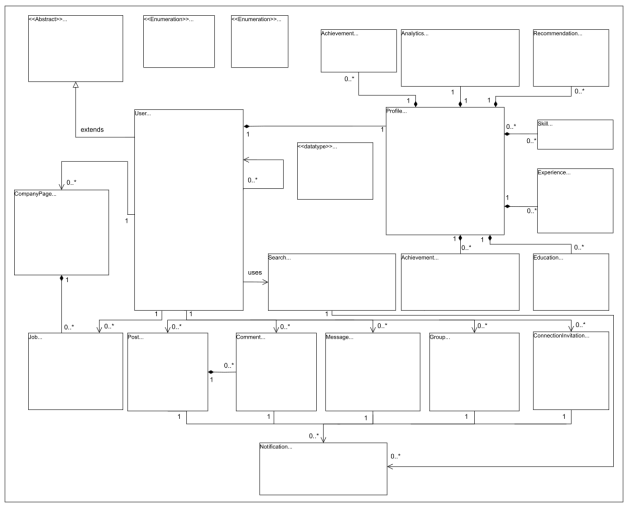
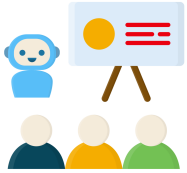


Figure 1.16: The class diagram of LinkedIn

- We know that a post or comment might need additional behaviors such as adding reactions, sharing, or threading replies. To extend these features without altering the base class, we can use the Decorator design pattern.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



We have completed the class diagram of LinkedIn according to the requirements. Now, let's design the sequence diagram of LinkedIn in the next lesson.

5 Sequence Diagram for LinkedIn

Sequence diagrams are an effective way to visualize the interactions between different entities and objects within a system. We can create different sequence diagrams for LinkedIn. In this lesson, we'll create sequence diagrams for the following interaction:

- **Send a connection invitation:** A user sends a connection invitation to another user.

5.1 Send a connection invitation

The sequence diagram for sending a connection invitation should have the following actors and objects that will interact with each other:

- **Actors:** `UserA` and `UserB`
- **Objects:** `Search`, `Notification`, and `ConnectionInvitation`

Here are the steps for sending a connection invitation interaction:

1. User A searches for User B using the search service.
2. If User B is found:
   (a) The search service returns User B.
   (b) A notification is sent to User B, indicating that they have appeared in a search.
   (c) User A creates a new connection invitation object for User B.
   (d) The connection invitation is sent from User A to User B.
   (e) A notification is sent to User B informing them of the connection request.

Based on the order above, the sequence diagram for sending a connection invitation on LinkedIn is given below:

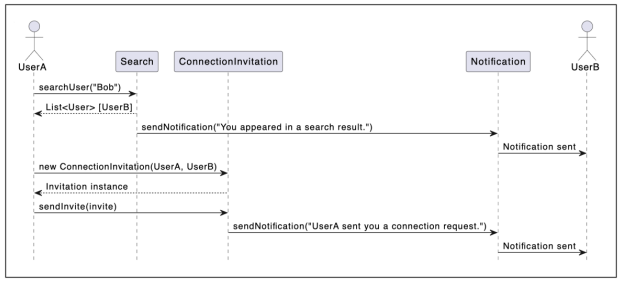


Figure 1.17: The sequence diagram for sending a connection invitation

Next, let's examine the activity diagrams for LinkedIn to understand the system's control flow.

6 Activity Diagram for LinkedIn

Activity diagrams are an effective way to visualize the flow of messages between activities within a system. We can create various activity diagrams for LinkedIn. In this lesson, we'll create activity diagrams for the following two activities:

- Creating a new post.
- **Activity challenge:** A user creates a page.

6.1 Creating a new post

The states and actions involved in this activity diagram are provided below.

States

Initial state: The user selects the new post option.
Final state: The post is published.

Actions

The user selects the "New Post" option, chooses the desired privacy setting, adds any necessary attachments, and then publishes the post.

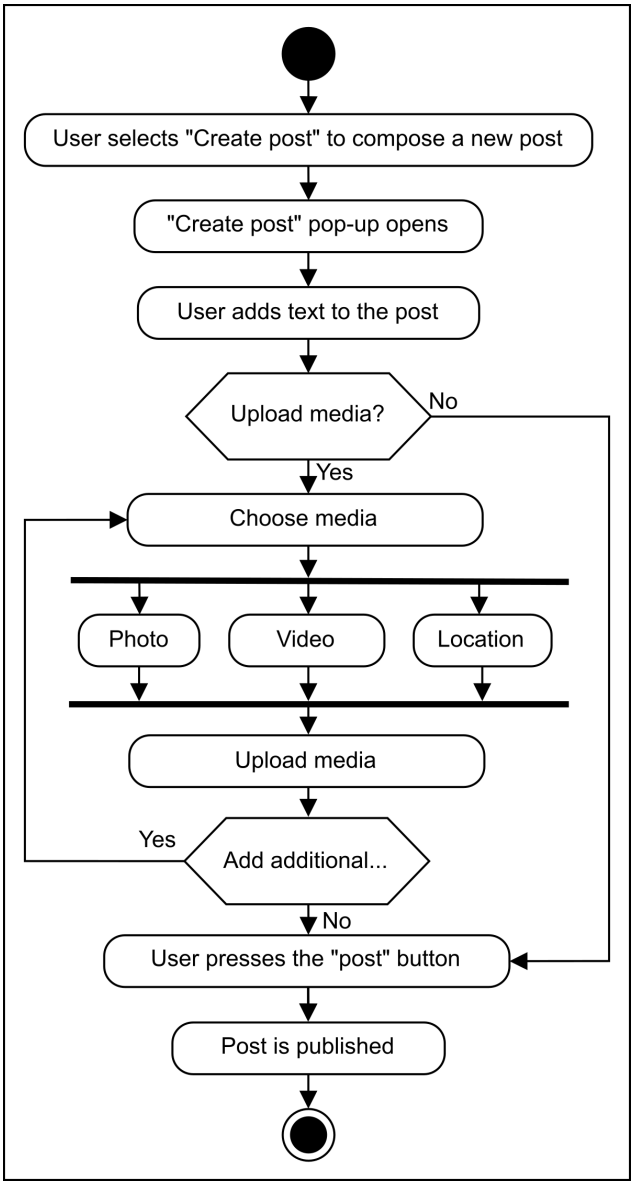


Figure 1.18: The activity diagram for a user creating a new post

6.2 Activity challenge: A user creates a company page

You'll help us create an activity diagram of a user creating a company page. A skeleton of the activity diagram is given below:

Notice that the actions in the diagram above are numbered from 1 to 10. The slots below represent the activities, and the arrows indicate the flow from one activity to another.

Can you rearrange the slots below in the correct order they should appear in the activity diagram given above?

Note: If you're stuck, click the "Show Solution" button.

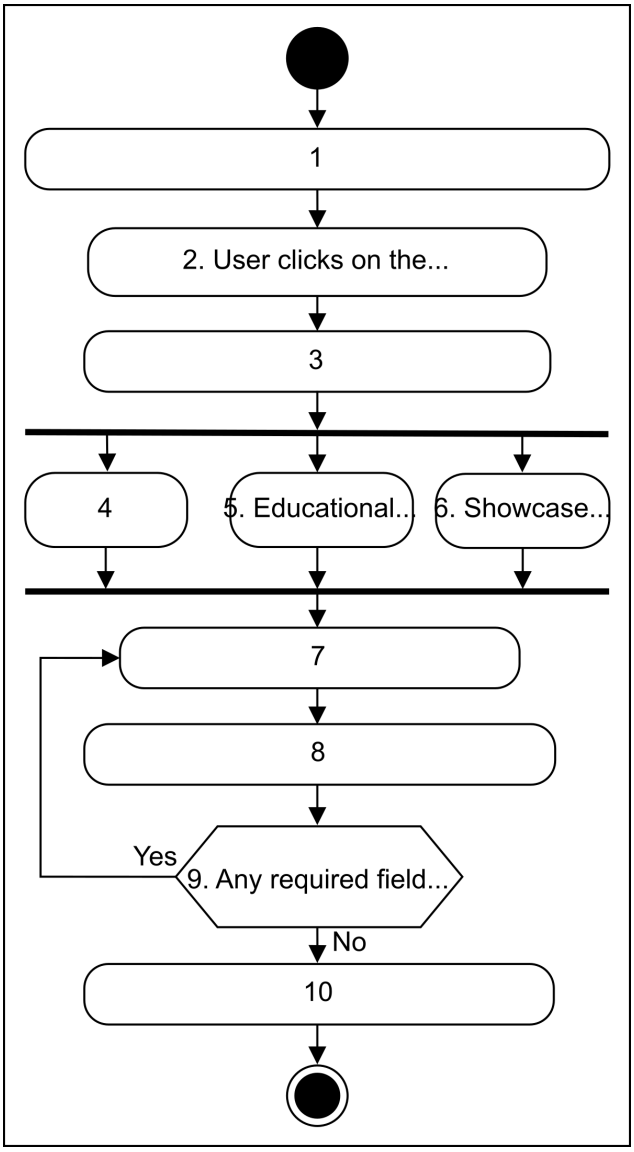


Figure 1.19: The activity diagram for a user creating a page

Alternatively, you can click the "Show complete diagram" button below to see the complete activity diagram.

We've reviewed some of LinkedIn's activity diagrams. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for LinkedIn

We've reviewed LinkedIn's various aspects and observed the attributes attached to the problem using various UML diagrams. Now , let's explore the more practical side of things. We will implement the LinkedIn network using multiple languages. This is usually the last step in an object-oriented design interview process. We have chosen the following languages to write the skeleton code of the different classes present in LinkedIn:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Designing LinkedIn.

7.1 LinkedIn classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Constants

The following code defines the various enums and custom data types being used in the LinkedIn design:

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Constant definitions

```
public class Address {
  private int zipCode;
  private String streetAddress;
  private String city;
  private String state;
  private String country;
}

enum ConnectionInviteStatus {
  PENDING,
  ACCEPTED,
  IGNORED
}

enum JobStatus {
```

```
  OPEN,
  ON_HOLD,
  CLOSED
}
```

Person and user

The `Person` class is an abstract class that contains details such as name, address, phone number, email, and password. It is derived from the `User` class.

The Person, Admin, and User classes

```
// Person will be an abstract class
public abstract class Person {
  private String name;
  private Address address;
  private String phone;
  private String email;
  private String password;
}

public class User extends Person {
  private int userId;
  private Date dateOfJoining;
  private Profile profile;
  private List<User> connections;
  private List<User> followsUsers;
  private List<CompanyPage> followCompanies;
  private List<Group> joinedGroups;
  private List<CompanyPage> createdPages;
  private List<Group> createdGroups;

  public boolean search(User user);
  public boolean sendMessage(Message message);
  public boolean sendInvite(ConnectionInvitation invite);
  public boolean cancelInvite(ConnectionInvitation invite);
  public Page createPage(CompanyPage page);
  public boolean deletePage(CompanyPage page);
  public Group createGroup(Group group);
  public boolean deleteGroup(Group group);
  public Post createPost(Post post);
  public boolean deletePost(Post post);
  public Comment createComment(Comment comment);
  public boolean deleteComment(Comment comment);
  public boolean react(Post post);
  public boolean requestRecommendation(User user);
  public boolean acceptRecommendation(User user);
  public boolean applyForJob(Job job);
}
```

Recommendation, achievement, and analytics

The `Recommendation`, `Achievement`, and `Analytics` classes will provide a user's personal information and make up the `Profile` class. The definition of these classes is given below:

The Recommendation, Achievement, and Analytics classes

```
public class Recommendation {
  private int userId;
  private Date createdOn;
  private String description;
```

```
  private boolean isAccepted;
}

public class Achievement {
  private String title;
  private Date dateAwarded;
  private String description;
}

public class Analytics {
  private int searchAppearances;
  private int profileViews;
  private int postImpressions;
  private int totalConnections;
}
```

Profile, experience, education, and skill

The `Experience`, `Education`, and `Skill` classes will provide a user's personal information and make up the `Profile` class. The definition of these classes is given below:

The Experience, Education, Skill, and Profile classes

```
public class Experience {
  private String title;
  private String company;
  private Address location;
  private Date startDate;
  private Date endDate;
  private String description;
}

public class Education {
  private String school;
  private String degree;
  private Date startDate;
  private Date endDate;
  private String description;
}

public class Skill {
  private string name;
}

public class Profile {
  private String headline;
  private String about;
  private String gender;
  private List<byte> profilePicture;
  private List<byte> coverPhoto;

  private List<Experience> experiences;
  private List<Education> educations;
  private List<Skill> skills;
  private List<Achievement> achievements;
  private List<Recommendation> recommendations;
  private Analytics analytics;

  public boolean addExperience(Experience experience);
  public boolean addEducation(Education education);
  public boolean addSkill(Skill skill);
  public boolean addAchievement(Achievement achievement);
}
```

Company, job, and group

LinkedIn users can create groups and company pages. The company page contains information about the company. The company pages will host various job postings. The `Job`, `CompanyPage`, and `Group` classes are shown below:

The Job, CompanyPage, and Group classes

```
public class Job {
  private int jobId;
  private String jobTitle;
  private Date dateOfPosting;
  private String description;
  private CompanyPage company;
  private String employmentType;
  private Address location;
  private JobStatus status;
}

public class CompanyPage {
  private int pageId;
  private String name;
  private String description;
  private String type;
  private int companySize;
  private User createdBy;
  private List<Job> jobs;

  public boolean createJobPosting();
  public boolean deleteJobPosting(Job job);
}

public class Group {
  private int groupId;
  private String name;
  private String description;
  private int totalMembers;
  private List<User> members;

  public boolean updateDescription();
}
```

Post, comment, message, and connection invitation

LinkedIn users can create posts and comments. They can also send messages and connection invitations to other users. The definition of `Post`, `Comment`, `Message`, and `ConnectionInvitation` classes is given below:

The Post, Comment, Message, and ConnectionInvitation classes

```
public class Post {
  private int postId;
  private User postOwner;
  private String text;
  private List<byte> media;
  private int totalReacts;
  private int totalShares;
  private List<Comment> comments;

  public boolean updateText();
}

public class Comment {
  private int commentId;
  private User commentOwner;
  private String text;
```

```
  private int totalReacts;
  private List<Comment> comments;

  public boolean updateText();
}

public class Message {
  private int messageId;
  private User sender;
  private List<User> recipients;
  private String text;
  private List<byte> media;

  public boolean addRecipients(List<User> recipients);
}

public class ConnectionInvitation {
  private User sender;
  private User recipients;
  private Date dateCreated;
  private ConnectionInviteStatus status;

  public boolean acceptConnection();
  public boolean rejectConnection();
}
```

Search and notification

The `Search` class contains information on users, company pages, groups, and jobs. It enables the search functionality based on the given criteria (user, company page, group, and job keywords).
The `Notification` class is responsible for sending notifications to users about any new messages, comments, posts, or connection invitations via the built-in notification option.
The definition of these classes is given below:

The Search interface and Notification classes

```
public Search {

  public List<User> searchUser(String name) {
    // functionality
  }

  public List<CompanyPage> searchCompany(String name) {
    // functionality
  }

  public List<Job> searchJobs(String title) {
    // functionality
  }

  public List<Group> searchGroups(String name) {
    // functionality
  }
}

public class Notification {
  private int notificationId;
  private Date createdOn;
  private String content;

  public boolean sendNotification(Account account);
}
```

7.2 Executable code: Designing LinkedIn

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of a LinkedIn system. The main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for the respective languages. You can click the "Run" button to execute the code.

What does this code show

- **System Initialization:** The simulation begins by setting up key components of the LinkedIn system: two users (Alice and Bob) with their respective profiles and addresses. Supporting objects, such as `Search`, `ConnectionInvitation`, and `Notification` are also ready. This sets the stage for simulating common LinkedIn interactions such as searching for users, sending connection invites, and engaging with posts.

- **Scenario 1 (User searches and sends a connection invitation):** User A (Alice) searches for User B (Bob) using the search feature. When Bob is found, a notification is automatically sent to him indicating he appeared in a search. Alice then creates a connection invitation and sends it to Bob. Another notification is triggered to inform Bob about the new connection request.

- **Scenario 2 (Connection request is accepted):** User B (Bob) receives the connection invitation from Alice. He chooses to accept the request, which updates the connection status to "accepted." The system then sends a notification to Alice, confirming that her connection request was approved.

- **Scenario 3 (User interaction via post and comment):** User A (Alice) creates a new post announcing a professional update. User B (Bob) views the post and comments on it. A notification is automatically sent to Alice, informing her that Bob commented on her post.

  Hint: Try customizing the code!
  This code is designed not just for reading, but for experimentation. You can edit, extend, or modify any part of the example.

Designing LinkedIn

```java
import java.util.*;

public class Driver {

    public static void main(String[] args) {
        // Create common address
        Address address = new Address(12345, "123 Main St", "Cityville", "StateX", "CountryY");

        // Create user profiles
        Profile profileA = new Profile("Software Engineer", "Passionate about tech", "Male");
        Profile profileB = new Profile("Data Analyst", "Loves data", "Female");

        // Create users
        User userA = new User("Alice", address, "1234567890", "alice@example.com", "password", 1,
            new Date(), profileA);
        User userB = new User("Bob", address, "0987654321", "bob@example.com", "password", 2, new
            Date(), profileB);

        System.out.println(" Users created:");
        System.out.println("User A: " + userA.getName());
        System.out.println("User B: " + userB.getName());

        // ========================
        // Scenario 1: Search & Connection Request
        // ========================
        System.out.println("
 Scenario 1: User A searches for User B and sends a connection request");
```

```java
        Search search = new Search();
        List<User> foundUsers = search.searchUser("Bob");

        // Simulate search result
        foundUsers.add(userB);

        if (!foundUsers.isEmpty()) {
            System.out.println(" User B found in search results");

            // Auto-notify User B of being searched
            Notification searchNotif = new Notification(1001, "You appeared in a search result.");
            searchNotif.sendNotification(userB);

            // Send connection request
            ConnectionInvitation invite = new ConnectionInvitation(userA, userB);
            userA.sendInvite(invite);
            System.out.println(" Connection request sent from " + userA.getName() + " to " + userB.
                getName());

            // Auto-notify User B of connection request
            Notification inviteNotif = new Notification(1002, userA.getName() + " sent you a
                connection request.");
            inviteNotif.sendNotification(userB);
        }

        // ========================
        // Scenario 2: User B accepts the connection
        // ========================
        System.out.println("
 Scenario 2: User B accepts the connection request");

        ConnectionInvitation invitation = new ConnectionInvitation(userA, userB);
        boolean accepted = invitation.acceptConnection();

        if (accepted) {
            System.out.println(" " + userB.getName() + " accepted the connection request");
            Notification notifyAccepted = new Notification(1003, userB.getName() + " accepted your
                connection request.");
            notifyAccepted.sendNotification(userA);
        } else {
            System.out.println(" Connection request not accepted");
        }

        // ========================
        // Scenario 3: User A creates a post and User B comments on it
        // ========================
        System.out.println("
 Scenario 3: User A creates a post and User B comments on it");

        Post post = new Post(101, userA, "Excited to start my new job!");
        System.out.println(" " + userA.getName() + " posted: "" + post.getText() + """);

        Comment comment = new Comment(201, userB, "Congratulations!");
        post.setComments(new ArrayList<>());
        post.getComments().add(comment);
        System.out.println(" " + userB.getName() + " commented: "" + comment.getText() + """);

        // Auto-notify User A of comment
        Notification commentNotif = new Notification(1004, userB.getName() + " commented on your
            post.");
        commentNotif.sendNotification(userA);

        // Post summary
        System.out.println("
 Post Summary:");
        System.out.println("Post by: " + post.getPostOwner().getName());
        System.out.println("Text: " + post.getText());
        System.out.println("Total Comments: " + post.getComments().size());
        System.out.println("Comment by: " + comment.getCommentOwner().getName());
    }
}
```

7.3 Wrapping up

We've explored the complete design of LinkedIn in this chapter. We 've looked at how LinkedIn can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing LinkedIn

After completing the LinkedIn case study, reviewing and reinforcing your understanding is a good time. We review the main system requirements, outline the core classes along with their roles and relationships, and highlight the key design principles that were applied. We 'll also explore how different objects interact within the system and walk through the overall workflow to see how all the components fit together to deliver the intended functionality.

8.1 Key requirements

Here are the primary functional and non-functional requirements that define the scope of the LinkedIn system:

1. Users can create and update their profiles to include education, experience, achievements, and skills.
2. Users can search for other users, company pages, groups, and job listings.
3. Users can send, cancel, accept, or ignore connection requests.
4. Users can follow or unfollow other users and company pages without needing to connect their accounts.
5. Users can view analytics, including connections, profile views, post impressions, and search appearances.
6. Users can request and give recommendations.
7. Users can create, react to, share, and comment on posts and on existing comments.
8. Users can send and receive direct messages.
9. The system sends notifications for messages, connection requests, search appearances, and post activities.
10. Users can create and join groups, as well as create or follow company pages.
11. Company pages can list job openings, and users can apply for them.

8.2 System actors

The following actor interacts with the LinkedIn system:

- **User:** A professional who creates a profile, manages connections, interacts with content, applies for jobs, joins groups, and receives notifications.

8.3 Important classes and relationships

Below is a summary of the key classes, their responsibilities, and their primary relationships in the system:

Class	Description	Important Relationships

`Person`	Abstract base class containing personal details like name, address, and phone number.	Parent of `User`, composed with `Account`
`User`	Represents a LinkedIn user, supports profile, messaging, posting, connections, etc.	Inherits from `Person`, has one `Profile`, associates with `CompanyPage`, `Group`, `Job`, `Post`, `Comment`, `Message`, `ConnectionInvitation`, uses `Notification`, `Search`
`Profile`	Contains the user's education, experience, skills, achievements, and recommendations.	Composed of `Education`, `Experience`, `Skill`, `Achievement`, `Recommendation`, `Analytics`
`CompanyPage`	Represents a company's profile on LinkedIn.	Created by `User`, composed of `Job`
`Job`	Represents job postings.	Belongs to `CompanyPage`, posted by `User`, has `Address`
`Group`	Represents a professional group for users.	Created/joined by `User`
`Post`	Represents user-generated content.	Owned by `User`, composed of `Comment`, notifies `Notification`
`Message`	Direct messages are exchanged between users.	Sent by `User`, can have multiple recipients
`ConnectionInvitation`	Tracks connection request status.	Sent between two `User` instances, notifies `Notification`
`Notification`	Manages notifications for user activities and system events.	Notifies `User` about various actions
`Search`	Interface for searching users, companies, jobs, and groups.	Returns relevant objects to `User`
`Recommendation`	Stores recommendations requested or given by users.	Part of `Profile`, can be added via system action
`Account`	Stores login credentials and account info.	Composed within `Person`

8.4 Design highlights

Here are some of the notable design choices made in the LinkedIn system:

8.5 Design approach

The system is designed using a bottom-up approach, starting with basic units such as `User`, `Post`, and `ConnectionInvitation`, which are then composed into larger structures, including `Profile`, `CompanyPage`, and the overall LinkedIn platform.

8.6 Design principles

The following SOLID principles are evident in the LinkedIn system:

- **Single Responsibility principle (SRP):** Each class is focused on a single aspect. For example, `Profile` manages profile details, while `Notification` handles alerts.
- **Open/Closed principle (OCP):** New features (like new analytics metrics or notification types) can be added by extending existing classes.

- **Liskov Substitution principle (LSP):** Derived classes (e.g., `User` extending `Person`) can be used wherever the base class is expected.
- **Interface Segregation principle (ISP):** Search operations for users, jobs, companies, and groups are separated via the `Search` interface.
- **Dependency Inversion principle (DIP):** High-level modules like `User` depend on abstractions (`Search`, `Notification`) rather than concrete implementations.

8.7 Design patterns

The system applies several classic design patterns:

- **Factory pattern:** Used for centralized creation of objects such as posts, connection invitations, and job listings.
- **Observer pattern:** Enables automatic notifications when certain actions occur (e.g., connection requests, new messages).
- **Singleton pattern:** Ensures only one instance of services like `Notification` or `Search` exists system-wide.
- **Strategy pattern:** Enables flexible search logic for various entity types (users, jobs, companies, and groups).
- **Decorator pattern:** Used to extend the behaviors of posts and comments, such as adding reactions or sharing.

8.8 Object interactions

Key objects collaborate as follows to achieve core LinkedIn functionalities:

- **User search:** A user utilizes the `Search` service to find other users; if successful, the searched user receives a notification about appearing in the results.
- **Connection invitations:** Sending a connection request creates a `ConnectionInvitation` object, which notifies the recipient of the new request.
- **Content engagement:** Creating posts or comments triggers notifications to owners or followers; reactions and replies are similarly handled through the `Notification` system.
- **Messaging:** Sending a message results in a `Message` object being delivered to the recipient(s) with real-time notification.
- **Job and group actions:** Applying for jobs or joining groups updates the associated entities and may trigger relevant notifications.

8.9 System workflow

The primary user journeys and life cycle events are:

- **Profile management:** Users can create or update their profiles, adding details such as education, experience, skills, and achievements.
- **Discovery and connection:** Users search for others, send connection requests, and manage incoming requests with real-time notifications at each step.
- **Content creation and interaction:** Users publish posts, comment, share, and react, while receiving notifications about activity on their content.

- **Private messaging:** Users exchange direct messages and are notified of new communications.
- **Engagement with organizations:** Users create or follow company pages, list or apply for jobs, and join or create groups, with notifications facilitating all major updates and actions.

9 Key Insights and Tips: LinkedIn System

Well done on finishing the LinkedIn system case study! In this lesson, we'll deepen your understanding by highlighting common design challenges and sharing effective strategies for achieving interview success. You 'll also find a quick quiz to test your knowledge and a list of related case studies to help you master essential object-oriented design concepts.

9.1 Common mistakes

Avoiding these common mistakes will help you create a more reliable and scalable LinkedIn-like system:

1. **Misunderstanding user roles and permissions:** Confusing the roles and permissions of `Users` and `Admins` can lead to unauthorized access or security breaches. Ensure that `Users` have limited access to their data and can perform actions like sending connection invitations or applying for jobs. `Admins`, on the other hand, should have privileges to manage content and users, such as blocking/unblocking users or deleting groups.
2. **Overcomplicating profile and post management:** A lack of modularity in managing `Profiles`, `Posts`, and `Comments` can lead to data redundancy or inefficient data handling. Separate `Profile` details (like `Education`, `Experience`, and `Skills`) into independent classes and use composition to assemble them in the `Profile` class. Similarly, treat `Posts` and `Comments` as independent entities with clear relationships.
3. **Inefficient handling of connection requests:** Mismanagement of `ConnectionInvitation` statuses (Pending, Accepted, Ignored) can cause issues with network building. Implement a dedicated `ConnectionInvitation` class with proper status handling, ensuring users can easily send, accept, or reject invitations and triggering real-time notifications when actions occur.
4. **Not considering scalability for notifications:** As the user base grows, failing to manage notifications efficiently can lead to performance bottlenecks and slow updates. Use design patterns like *Observer* to efficiently manage and send notifications about `Messages`, `Posts`, `Comments`, and Connection Requests, ensuring that the system can scale as user activity increases.
5. **Lack of robust search and discovery features:** Failing to implement a scalable and efficient `Search` system for users, jobs, and groups can lead to a poor user experience. Design the `SearchCatalog` class to efficiently index and retrieve user profiles, groups, and job postings. Implement search filters based on user preferences, job titles, and other relevant criteria to improve discoverability.

9.2 Interview tips

Understand key design patterns, system components, and real-world applications to prepare for your interview and confidently discuss your approach to these concepts.

Tip	Explanation

Discuss extendability for new features.	Explain how you'd add future-ready modules: e.g., supporting a new post type, more analytics, or company verification—using patterns like Factory and Open/Closed Principle.
Explain design patterns.	Highlight design patterns such as Observer (for notifications), Singleton (for system-wide singletons), and Factory (for creating different types of objects). Explain why these patterns are chosen.
Focus on real-world applications.	Emphasize how LinkedIn handles millions of users and connections in real-time. Discuss challenges like scalability, performance optimization, and the handling of edge cases (e.g., network issues).
Understand the various types of user interactions.	Understand the differences between connection requests, messages, and post reactions. Be ready to explain how LinkedIn handles these interactions.
Explain edge-case handling.	For example, describe how your system handles duplicate connection requests, deactivated accounts, or a message delivery failure.

9.3 Test your understanding

Now that you understand OOD principles well, solve the quiz below to test your knowledge.

LinkedIn System

Question 1: What approach would you use if you want to add polls as a new post type without modifying existing logic?

- Add new methods to the base `Post` class.
- **[CORRECT]** Leverage the Factory pattern to create specific `Post` subclasses.
  Explanation: Factory pattern enables new post types via subclassing, supporting Open/Closed principle.
- Use global variables.
- Make all posts identical.

Question 2: What design pattern would you use to send notifications to users about various events (e.g., new message, post comment, etc.)?

- Factory
- Strategy
- Singleton

- **[CORRECT]** Observer
  Explanation: Observer pattern enables the system to notify all subscribed users or services whenever events like new messages or comments occur. This keeps notification logic flexible, decoupled, and easy to scale as the platform grows.

Question 3: What relationship should exist between `User` and `Profile` in the class diagram?

- Aggregation
- **[CORRECT]** Composition
  Explanation: A user "owns" a profile; if the user is deleted, the profile is removed too.
- Inheritance
- Association

Question 4: Which class would handle user posts, including the content and reactions?

- `Post`
- `Profile`
- `Job`
- **[CORRECT]** `Group`

Question 5: Which OOD principle allows you to add new filters to `SearchCatalog` without modifying its core implementation?

- **[CORRECT]** A. Open/Closed principle
  Explanation: New filters can be plugged in as extensions.
- Liskov Substitution principle
- Interface Segregation principle
- YAGNI principle

9.4 Related case studies

The following case studies will help reinforce concepts used in the LinkedIn system:

- **Designing Facebook:** Explores designing scalable news feeds, notifications, user profiles, comments, and real-time interactions—concepts directly applicable to LinkedIn event feeds and user engagement.
- **Design of a social media platform:** Focuses on designing a platform similar to Facebook or Instagram, managing user profiles, posts, connections, and notifications in a scalable and efficient manner.
- **Design of a job portal:** This case study covers the design of a system for job seekers and employers, focusing on job applications, job postings, and job search functionality.
- **Design of a messaging system:** Explores how to build a robust messaging system that handles one-to-one and group communication, notification triggers, and real-time updates.

- **Design of a professional recommendation system:** Shows how endorsements, analytics, and recommendations can be integrated for social proof and networking.